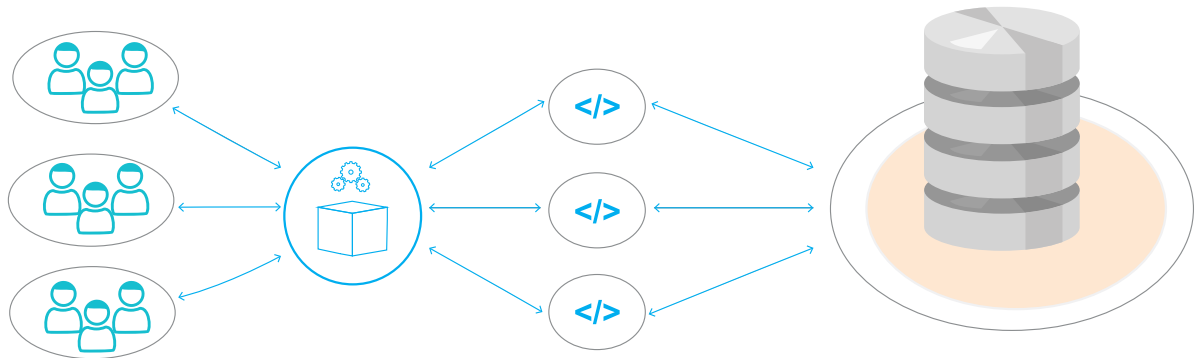


Demystifying Connection Pools: A Deep Dive

Connection pooling can be a game changer for your app's performance. This final part of the two-part series of articles on connection pooling continues to uncover its magic.



In the first part of this series, we covered how typical connections are established, what connections pools are and how they work. We also started implementing a connection pool of our own, and figured out the basic building blocks needed to do so. Let's continue from where we left off.

We have already implemented the Get, Add and Close methods for our connection pool. Now, we need to tie them together and provide an interface to create a connection pool. We will implement a `NewConnectionPool` function as a constructor for a new connection pool. It takes the driver, `dataSource`, and `maxSize` arguments and returns a pointer to a new `ConnectionPool` instance. It first checks if the provided driver and `dataSource` arguments are valid by opening a connection to the database. If the connection is successful, it initialises a new connection pool with the provided `maxSize` argument. It then creates a new channel of `*sql.DB` objects and pre-populates it with `maxSize` database connections by creating a new database connection for each iteration of a loop. Finally, it returns the new `ConnectionPool` instance.

```
func NewConnectionPool(driver, dataSource string, maxSize
int) (*ConnectionPool, error) {
```

```
    // Validate driver and data source
    _, err := sql.Open(driver, dataSource)
    if err != nil {
        return nil, err
    }
```

```
    cp := &ConnectionPool{
        queue:    make(chan *sql.DB, maxSize),
        maxSize:  maxSize,
        currentSize: 0,
    }
```

```
    cp.isNotEmpty = sync.NewCond(&cp.lock)
    cp.isNotFull = sync.NewCond(&cp.lock)
```

```
    for i := 0; i < maxSize; i++ {
        conn, err := sql.Open(driver, dataSource)
        if err != nil {
            return nil, err
        }
        cp.queue <- conn
        cp.currentSize++
    }
```

```
    return cp, nil
}
```

Putting it all together

This is what our final custom `ConnectionPool` implementation looks like:

```
package pool
```

```
import (
    "database/sql"
```